Practical Task -

Task 1: Multi-Tenant Invitation System

Goal: Design an API to invite users to a multi-tenant platform and send them an email.

Requirements:

Create an `Invitation` model with: `name`, `email`, `tenant`, `status`, `token`, `expiration_date`.

Implement API to:

- Create invitation (set `status=Pending`, generate `token`)
- Accept invitation (token-based, set password)
- Cancel invitation

Set expiration date to 7 days later.

Write custom Django command or Celery task to mark expired invitations as `"Expired"`.

Bonus: Store metadata (browser IP or note field) when sending the invite.

Tasks 2: Permission-Aware Endpoint Decorator

Goal: Enforce permission checks using product-feature-role mapping.

Requirements:

Create a decorator:

```
@check_permission(product_id="abc", feature="dashboard", permission="read")
```

On request:

- Resolve `request.user`, `tenant`, `role`, and allowed features.
- Deny access if permission doesn't exist.

Bonus:

Integrate with centralized Auth permission API.

Task 3: Implement Centralized Logging Across Microservices

Objective

Design and implement a centralized logging system for a distributed Django microservices architecture. Each service should forward structured logs to a centralized logging backend (like Grafana or ELK stack).

Requirements

- **Logging Format**
- Each log entry must include:

Timestamp
Service name
Request ID (trace ID for distributed tracing)

Log level
Message
Optional: User ID, Tenant ID

- **Microservice Integration**
  Integrate logging setup in at least **2 Django services** (e.g., `auth` and `tenant`).
  Use Python's `logging` module with JSONFormatter (like `python-json-logger`).
- **Trace ID Propagation**
  Generate a unique trace ID at the entry point (e.g., API Gateway or Nginx).
  Propagate the trace ID through Django middleware and HTTP headers across services.
- **Central Logging Backend (Simulated)**
  Set up **Grafana Loki (preferred)** or **ELK (Elasticsearch, Logstash, Kibana)** via Docker.
  Send logs via **Promtail** (for Loki) or **Filebeat** (for ELK).
  Demonstrate search capability in Grafana/Kibana dashboards.
- **Bonus: Correlation View**
  Implement a dashboard or view where logs can be filtered by:
  Service
  Tenant ID
  Trace ID

Task 4: Circuit Breaker in Django Middleware

Goal:

Prevent your Django app from repeatedly calling a failing external service.

Requirements:

Create a custom middleware `ExternalServiceCircuitBreakerMiddleware`.
Monitor failed responses from selected domains (e.g., Auth, Billing API).
If a domain returns 5xx errors more than 3 times within a 1-minute window:
Open the circuit and block outbound requests to that domain for 2 minutes.
Use Django's cache (or Redis) to persist circuit states.
Log circuit transitions and blocked request attempts.